

Sistemas Embebidos

Trabajo práctico N°3 - Año 2025

Temporización - Network Time Protocol (NTP) - Memoria EEPROM

Objetivos

- Leer y escribir memoria EEPROM.
- Utilizar servicios disponibles en la nube.
- Implementar aplicaciones de IoT completas.

Metodología

Trabajo individual o grupal. 2 estudiantes por grupo máximo.

Tiempo de realización estimado: 3 o 4 clases.

Aprobación

- Mostrar en clases la aplicación funcionando correctamente.
- Enviar los programas de computación implementados a través de la plataforma Moodle.
- Escribir un breve informe y enviarlo a través de la plataforma Moodle.

Materiales necesarios

- Placas Arduino UNO y circuito con leds sensores (provistas por la cátedra).
- Entorno de desarrollo de Arduino UNO (puede descargarse de <https://www.arduino.cc/en/software>).
- Intérprete de lenguaje de programación Python 3.
- Módulo pySerial para la comunicación serial (ver apunte sobre PySerial en aula abierta).
- Servidor web. Puede utilizar:
 - Frameworks de desarrollo web. Se sugiere Flask, FastAPI o Django (en el aula abierta encontrará un apunte sobre Flask).
 - Apache Web Server (en el trabajo práctico N°6 de la asignatura Redes de Computadoras se dan instrucciones de instalación y uso).
- Módulos ntplib (Network Time Protocol para Python) y ctime. ntplib no viene por defecto en Python3. Puede instalarse con `pip3 install ntplib` (en el aula abierta encontrará un apunte sobre NTP).
- **Opcional:** Librerías para Arduino IDE FreeRTOS de Richard Barry. Verificar que diga “implementado para AVR (Uno, Nano, Leonardo, Mega)”.
- **Opcional:** Puede escribir el código a ejecutar en el Arduino UNO empleando un simulador adecuado. La cátedra provee una simulación con el circuito listo para ser utilizado en el enlace: <https://wokwi.com/projects/391281841803809793>

Chatbots de IA sugeridos:

Si bien todos los chatbot de IA son adecuados como soporte para resolver este trabajo práctico, la cátedra ha verificado que las siguientes son adecuadas:

- ChatGPT de OpenAI (<https://chatgpt.com/>). Muy adecuado.
- Gemini de Google (<https://gemini.google.com/>). Ofrece buenas explicaciones y ejemplos, adecuado para aprender. Pero comete errores, algunas veces errores significativos.
- MetaAI de Meta (accesible a través de Whatsapp). No es adecuado si se usa FreeRTOS. Fuera de FreeRTOS presenta un comportamiento satisfactorio.

Actividades

Escribir una aplicación que realice las siguientes tareas:

1. Implemente en el microcontrolador un reloj en tiempo real en formato Unix, es decir, que cuente la cantidad de segundos desde el inicio de tiempos de sistemas Unix (1/1/1970 00:00:00 tiempo UTC). Utilice las funciones de la librería **ctime** de Python para manejar formatos de representación de hora.
 - a. Al iniciar la aplicación, el backend en Python deberá obtener la hora mediante el protocolo NTP (Network Time Protocol) y transmitir al Arduino la cantidad de segundos desde el origen de tiempos Unix. Puede utilizar la librería **ntplib** de Python u obtener la hora directamente del sistema Linux
 - b. El microcontrolador solo deberá incrementar una unidad cada segundo. Deberá utilizar una interrupción o tarea de alta prioridad si utiliza FreeRTOS. Tenga en cuenta que necesitará una variable de varios bytes para almacenar los segundos necesarios (por ejemplo: long int). Si utiliza la FreeRTOS, no puede usar la función vTaskDelay, porque entre ejecuciones de vTaskDelay se perderían fracciones de segundo. En su lugar, debe utilizar vTaskDelayUntil(TickType_t *pxPreviousWakeTime, TickType_t *xTimeIncrement); que almacena automáticamente en la variable pxPreviousWakeTime la cantidad de ticks desde la última ejecución de vTaskDelayUntil.
 - c. La página web debe incluir un botón “Actualizar hora desde servidor NTP” que actualice la hora usando el protocolo NTP.
2. Implementar interrupciones en los pines 2 y 3 del Arduino Uno, de modo que cuando se presionen los pulsadores conectados a dichos pines, se almacene en la memoria EEPROM la hora UTC completa a la cual ocurrió el evento y el evento ocurrido. La función debe ser tal que, si el Arduino se desconecta de la fuente de alimentación y se vuelve a conectar, nuevas escrituras en la EEPROM no deben sobrescribir escrituras anteriores.
3. Implementar funciones que permitan recuperar la información almacenada en la EEPROM para mostrarse en la página web cuando se presione un botón

desde la misma. En la página web se deberán mostrar los eventos ocurridos y la hora UTC en formato legible (utilice la librería **ctime**).

4. La memoria EEPROM debe poder borrarse completamente cuando se presione un botón implementado en la página web.
5. Toda la interacción del usuario con la aplicación debe realizarse a través de una página web (puede reutilizar los mecanismos implementados en trabajos prácticos anteriores).

Opciones de implementación

- Puede utilizar FreeRTOS o no. FreeRTOS permitirá un código mucho más ordenado, pero puede tener problemas con el tamaño de la pila.
- Para el reporte de datos del backend a la página web, puede ser adecuado el uso de websocket o mecanismos para el envío de mensajes como JSON, AJAX o similares.

Anexo 1: Leer y escribir la memoria EEPROM

Cada posición de la memoria EEPROM almacena un byte. Se sugiere analizar los ejemplos `eeeprom_clear`, `eeeprom_write` y `eeeprom_read` para comprender el funcionamiento de la memoria EEPROM. Estos ejemplos se encuentran en la carpeta de ejemplos EEPROM, instalada por defecto en Arduino IDE.

Considerar que la hora requiere una variable de varios bytes, mientras que cada posición de la memoria EEPROM almacena un byte.

Tener en cuenta que la escritura de la memoria flash para cargar un nuevo programa no sobrescribe la memoria EEPROM. Cuando una posición de la memoria EEPROM está vacía, tiene escrito el número binario 11111111 (0xFF).

Sugerencia: Debe crear una función cuyo propósito sea solo recibir datos. Los datos deben ser leídos solo cuando llegaron la totalidad de los caracteres del string. Para interpretar el comando que llegó, crear una tarea diferente.